



**INSTITUTO DE CIÊNCIA E TECNOLOGIA DE SOROCABA
ENGENHARIA DE CONTROLE E AUTOMAÇÃO**

**Projeto de Estrutura de Dados - SISTEMA DE MONITORAMENTO DE
ENFERMEIRAS NA PANDEMIA**

William Santos Moreira

Sorocaba

2026

Sumário

1. Introdução.....	2
2. Objetivo.....	2
3. Fundamentação Teórica.....	2
3.1. Filas.....	3
3.2. Tabelas Hash.....	3
3.3. Árvores AVL.....	3
3.4. KD-Tree.....	4
3.5. Segment Tree.....	4
4. Descrição do Dataset.....	4
5. Desenvolvimento do Sistema.....	5
5.1. Arquitetura Geral.....	5
5.2. Utilização da I.A.....	6
6. Implementação das Estruturas de Dados.....	6
6.1. Fila.....	6
6.2. Tabela Hash.....	6
6.3. Árvore AVL.....	7
6.4. AVL Otimizada.....	7
6.5. KD-Tree.....	7
6.6. Segment Tree.....	7
7. Funcionalidades Implementadas.....	7
7.1. Inserção de Dados.....	7
7.2. Busca de Dados.....	7
7.3. Remoção de Dados.....	8
7.4. Cálculos Estatísticos.....	8
7.5. Detecção de Anomalias.....	8
7.6. Monitoramento de Burnout.....	8
7.7. Consultas Espaciais.....	8
7.8. Consultas Temporais.....	8
8. Otimização da Estrutura AVL.....	8
9. Resultados e Análise dos Benchmarks.....	9
9.1. Desempenho em Cenário Ideal (Baseline).....	13
9.2. Restrição de Memória.....	13
9.3. Restrição de processamento.....	13
9.4. Restrição de Latência.....	13
9.5. Restrição de dados (Sensores Corrompidos).....	13
9.6. Restrição Algorítmica (XOR).....	13
9.7. Comparação entre AVL Tradicional e AVL Otimizada.....	14
10. Discussão dos Resultados.....	14
11. Conclusão.....	15
12. Referências.....	15

1. Introdução

Na área da saúde, é cada vez mais comum a utilização de dispositivos vestíveis que permitem coletar dados fisiológicos, como frequência cardíaca, temperatura da pele e atividade eletrodérmica, gerando grandes quantidades de informações que precisam ser armazenadas e processadas de forma eficiente. Neste trabalho foi desenvolvido um sistema em linguagem C para simular o processamento contínuo desses dados biométricos. O sistema utiliza um dataset contendo 11,5 milhões de registros de monitoramento de enfermeiras durante a pandemia e realiza operações de inserção, busca, remoção e análise dos dados, simulando um ambiente de monitoramento em tempo real.

Para atender aos requisitos da disciplina, foram implementadas cinco estruturas de dados a esse programa: fila, tabela hash, árvore AVL, KD-Tree e Segment Tree. Cada estrutura possui uma função específica dentro do sistema, permitindo organizar os dados de maneira eficiente e realizar diferentes tipos de consultas, análises e cálculos estatísticos que podem identificar possíveis situações de estresse elevado e início de burnouts. Além de avaliar o desempenho das estruturas implementadas por meio de benchmarks realizados em diferentes condições. Buscando mostrar na prática a importância de escolhas adequadas das estruturas usadas para armazenar grandes quantidades de dados.

2. Objetivo

O objetivo deste trabalho é desenvolver um sistema em linguagem C capaz de simular o processamento contínuo de dados biométricos em tempo real, utilizando diferentes estruturas de dados para realizar operações de inserção, busca, remoção e análise das informações. Além disso, busca-se comparar o desempenho das estruturas implementadas por meio de benchmarks, avaliando aspectos como tempo de execução, consumo de memória, escalabilidade e eficiência em diferentes condições de funcionamento, demonstrando na prática a aplicação dos conceitos estudados na disciplina de Estrutura de Dados.

3. Fundamentação Teórica

Como solicitado para este trabalho foram escolhidas 5 estruturas de dados, sendo três estruturas tradicionais e apresentadas na ementa (Fila, Hash, Árvore AVL) e as outras duas das quais não fazem parte da ementa (KD-Tree e Segment Tree) cada uma tendo sua função específica. Abaixo temos as funções de cada uma explicadas.

3.1. Filas

A fila é uma estrutura de dados linear onde o primeiro elemento inserido é também o primeiro a ser removido. As operações básicas dessa estrutura são inserção e remoção.

Neste projeto, a fila é utilizada para simular a chegada contínua dos dados dos sensores biométricos, representando um ambiente de monitoramento em tempo real. Todos os registros do dataset passam inicialmente pela fila antes de serem distribuídos para as outras estruturas de dados do sistema.

3.2. Tabelas Hash

A tabela hash é uma estrutura de dados utilizada para armazenar e recuperar informações de forma rápida. Seu funcionamento baseia-se em uma função hash, responsável por transformar uma chave em um índice de um vetor, permitindo acesso aos dados.

Quando diferentes chaves geram o mesmo índice, ocorre uma colisão, que pode ser tratada por diferentes técnicas. Neste trabalho foi utilizado o encadeamento, em que os elementos com a mesma posição na tabela são armazenados em uma lista ligada. No sistema por conta da pesquisa por datas ocorreram diversas colisões, pois o sistema buscava por um só identificador, a data, e encontrava vários dados na mesma.

3.3. Árvores AVL

É uma árvore binária de busca auto balanceada. Sua principal característica é manter a diferença de altura entre as subárvores esquerda e direita de qualquer nó em, no máximo, uma unidade. Sempre que esse limite é ultrapassado, rotações são realizadas para restaurar o balanceamento da árvore. Graças a esse mecanismo, as operações de inserção, busca e remoção apresentam complexidade $O(\log n)$, mesmo com grandes volumes de dados.

Neste trabalho foram utilizadas duas versões da árvore AVL. Uma implementação tradicional e outra otimizada. A versão otimizada armazena informações adicionais em cada nó, como a quantidade de elementos e a soma dos valores de frequência cardíaca, permitindo a realização de cálculos estatísticos de forma mais eficiente quando comparada à implementação convencional.

3.4. KD-Tree

A KD-Tree (K-Dimensional Tree) é uma estrutura de dados utilizada para organizar informações em espaços multidimensionais. Diferentemente das árvores binárias tradicionais, cada nível da árvore utiliza uma dimensão diferente como critério de comparação, permitindo representar dados com duas, três ou mais dimensões.

No sistema desenvolvido, a KD-Tree organiza os valores dos eixos X, Y e Z obtidos pelos sensores de movimento. Isso permite realizar buscas por registros com padrões de movimentação semelhantes, auxiliando na identificação de comportamentos físicos associados aos níveis de estresse monitorados.

3.5. Segment Tree

A Segment Tree é uma estrutura de dados projetada para realizar consultas eficientes sobre intervalos de valores. Ela pode calcular operações como soma, média, mínimo ou máximo em determinados intervalos do conjunto de dados.

Neste projeto, a Segment Tree foi utilizada para calcular médias móveis dos sinais biométricos, como frequência cardíaca e atividade eletrodérmica, permitindo analisar a evolução dos dados ao longo do tempo. Essas informações foram usadas pelo sistema para identificar tendências relacionadas ao aumento do estresse e possíveis situações de burnout.

4. Descrição do Dataset

O dataset escolhido tem diversos dados sobre a identificação de estados de estresse, coletada em contexto hospitalar real durante o período da pandemia de COVID-19, consolidando as informações biométricas obtidas de enfermeiras em campo.

Os dados foram coletados ao longo de uma semana durante os turnos regulares de 15 enfermeiras, com idades entre 30 e 55 anos. Foram excluídas participantes com gravidez, tabagismo intenso, transtornos mentais ou doenças crônicas e cardiovasculares. A coleta ocorreu em fases, com destaque para a Fase-I (abril a agosto de 2020) e a Fase-II (outubro a dezembro de 2020). Foram monitoradas continuamente métricas fisiológicas essenciais, como atividade eletrodérmica (EDA), frequência cardíaca (HR) e temperatura da pele, com foco também em Resposta Galvânica da Pele e Pulso de Volume Sanguíneo, como mostrado na Tabela 1.

Variável	Sensor
X, Y, Z	Acelerômetro triaxial (MEMS)
EDA	Sensor GSR (Galvanic Skin Response)
HR	Sensor óptico PPG (Photoplethysmography/BVP)
TEMP	Sensor de temperatura de contato (termistor)
id	Identificador do participante
datetime	Timestamp do dispositivo
label	Questionário de autorrelato de estresse via smartphone

Tabela 1 – Variáveis do dataset utilizado.

O banco de dados resultante dessa coleta de dados, disponível no Kaggle, contém um arquivo CSV com cerca de 11,5 milhões de entradas distribuídas em nove colunas, o que trouxe um grande desafio para o desenvolvimento do projeto.

5. Desenvolvimento do Sistema

5.1. Arquitetura Geral

O sistema foi desenvolvido na linguagem C, com a ajuda de diversas inteligências artificiais como o Gemini e o ChatGpt. Inicialmente, os registros do arquivo CSV são lidos e inseridos em uma fila encadeada, que simula a chegada contínua dos dados em um ambiente de monitoramento em tempo real. Em seguida, cada registro é distribuído para as demais estruturas implementadas: tabela hash, árvore AVL, KD-Tree e Segment Tree. Cada estrutura possui uma finalidade distinta dentro do sistema que será explicada em tópicos mais a frente, e após guardar todos os dados o sistema apresenta

uma tela via terminal apresentada na Figura 1. O sistema conta também com benchmark usado para testar as estruturas.

```
=====
      SISTEMA DE MONITORAMENTO DE ENFERMEIRAS PANDEMIA
=====
Enfermeira: [Nenhuma] | Medicoes Avaliadas: 0
-----
1. CARREGAR Lote da Enfermeira via Fila IoT (Queue)
2. BUSCAR Registro por Data Exata (Hash O(1))
3. BUSCAR Medicoes com Padroes de Movimento Semelhantes (KD-Tree)
4. CONSULTAR Medias Moveis Sinais Vitais (Segment Tree)
5. DIAGNOSTICO: Picos de Estresse (Fisico vs Mental)
6. DIAGNOSTICO: Quase Burnout e Burnout
7. SIMULADOR: Benchmark e Escalabilidade de Memoria
8. Sair
Escolha uma opcao: [ ]
```

Figura 1 – Tela inicial do sistema.

Após o processamento, o sistema permite realizar buscas, análises estatísticas, identificar situações de estresse e executar benchmarks para avaliar o desempenho das estruturas de dados.

5.2. Utilização da I.A

Como permitido, a IA foi empregada diversas vezes no desenvolvimento do código em C, sendo muitas vezes utilizada para preencher lacunas de conhecimento e como um meio de aquisição de informações e retirada de dúvidas. Mas foi amplamente utilizada no tratamento de erros, na formatação das saídas, na otimização da árvore AVL e principalmente na criação dos testes de benchmark

6. Implementação das Estruturas de Dados

6.1. Fila

A fila foi implementada utilizando uma lista encadeada, permitindo a inserção e remoção de elementos de forma eficiente. Sua principal função é simular a chegada contínua dos dados dos sensores, fazendo com que todos os registros do arquivo CSV passem primeiro pela fila antes de serem enviados para as demais estruturas do sistema.

6.2. Tabela Hash

A tabela hash foi implementada para armazenar os registros indexados pela data de coleta. A partir de uma função hash, cada data é convertida em uma posição da tabela, permitindo consultas rápidas aos registros. Para tratar possíveis colisões, foi utilizado o método de encadeamento, no qual múltiplos registros podem ser armazenados na mesma posição por meio de listas ligadas.

6.3. Árvore AVL

A árvore AVL foi utilizada para organizar os registros de acordo com a frequência cardíaca (HR). Durante as inserções e remoções, a árvore realiza rotações automaticamente para manter seu balanceamento, garantindo que as operações de busca, inserção e remoção permaneçam eficientes mesmo com grandes quantidades de dados.

6.4. AVL Otimizada

Além da implementação tradicional, foi desenvolvida uma versão otimizada da árvore AVL. Nessa versão cada nó armazena informações adicionais, como a quantidade de elementos e a soma das frequências cardíacas da subárvore. Dessa forma, operações de cálculo estatístico podem ser realizadas com maior eficiência, reduzindo o tempo de processamento quando comparadas à árvore AVL convencional.

6.5. KD-Tree

A KD-Tree foi implementada para organizar os dados de aceleração dos sensores nos eixos X, Y e Z. Essa estrutura permite localizar registros com padrões de movimentação semelhantes, sendo utilizada para comparar diferentes comportamentos físicos registrados durante o monitoramento.

6.6. Segment Tree

A Segment Tree foi implementada para realizar consultas em intervalos de dados. Ela armazena informações necessárias para calcular médias móveis da frequência cardíaca e da atividade eletrodérmica de forma eficiente. Essas consultas são utilizadas pelo sistema para acompanhar a evolução dos sinais biométricos e auxiliar na identificação de situações de estresse e risco de burnout.

7. Funcionalidades Implementadas

7.1. Inserção de Dados

A inserção dos dados é realizada a partir da leitura do arquivo CSV. Cada registro é armazenado inicialmente na fila e, em seguida, distribuído para as demais estruturas de dados do sistema, permitindo que todas permaneçam sincronizadas durante o processamento.

7.2. Busca de Dados

O sistema permite realizar buscas utilizando diferentes estruturas de dados. A tabela hash é utilizada para localizar registros por data, enquanto a árvore AVL realiza buscas com base na frequência cardíaca. A KD-Tree é utilizada para encontrar registros com padrões de movimentação semelhantes.

7.3. Remoção de Dados

O sistema também permite remover registros armazenados na tabela hash a partir da data informada pelo usuário. Essa funcionalidade possibilita excluir conjuntos de registros de forma rápida e eficiente.

7.4. Cálculos Estatísticos

São realizados cálculos estatísticos utilizando os dados armazenados nas estruturas implementadas. Entre eles estão médias de frequência cardíaca e atividade eletrodérmica, utilizadas para acompanhar o comportamento dos sinais biométricos ao longo do tempo.

7.5. Detecção de Anomalias

O sistema monitora continuamente os valores de frequência cardíaca e atividade eletrodérmica para identificar possíveis situações de estresse. Quando esses valores ultrapassam limites previamente definidos, o sistema registra um alerta de estresse agudo.

7.6. Monitoramento de Burnout

Além da detecção de eventos isolados, o sistema realiza uma análise do comportamento dos sinais ao longo do tempo. A partir das médias móveis calculadas pela Segment Tree, é possível identificar aumentos persistentes dos níveis de estresse, indicando um possível risco de burnout.

7.7. Consultas Espaciais

As consultas espaciais são realizadas por meio da KD-Tree, que compara os valores de aceleração nos eixos X, Y e Z. Essa funcionalidade permite localizar registros com padrões de movimentação semelhantes aos informados pelo usuário.

7.8. Consultas Temporais

As consultas temporais são realizadas utilizando a Segment Tree, permitindo calcular médias em diferentes intervalos de tempo de forma eficiente. Essa funcionalidade é utilizada para acompanhar a evolução dos sinais biométricos e auxiliar nas análises realizadas pelo sistema.

8. Otimização da Estrutura AVL

Para melhorar o desempenho do sistema, foi desenvolvida uma versão otimizada da árvore AVL além da implementação tradicional. Enquanto a AVL convencional armazena apenas os dados necessários para manter a árvore balanceada, a versão otimizada também armazena informações adicionais em cada nó, como a quantidade de elementos e a soma dos valores de frequência cardíaca da subárvore.

Com esses dados armazenados, o sistema consegue realizar cálculos estatísticos de forma mais rápida, sem a necessidade de percorrer toda a árvore a cada consulta. Isso reduz o tempo de processamento e torna a execução mais eficiente, principalmente quando há um grande volume de registros.

A comparação entre a AVL tradicional e a AVL otimizada foi realizada

por meio dos benchmarks do sistema, permitindo avaliar os ganhos de desempenho obtidos com essa otimização. Os resultados mostram que a inclusão de metadados nos nós aumenta um pouco o uso de memória, porém proporciona consultas estatísticas significativamente mais rápidas, tornando a estrutura mais adequada para aplicações que processam grandes quantidades de dados em tempo real.

9. Resultados e Análise dos Benchmarks

Abaixo temos as telas dos testes de benchmark sequencialmente apresentadas. Na Figura 2 temos o menu onde é possível escolher o teste que será feito podendo ser nas configurações ideais ou com limitações.

```
=====
LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintotica)
=====
0. Baseline (Condicoes Ideais)
1. Restricao de MEMORIA (Max 500 nos)
2. Restricao de PROCESSAMENTO
3. Restricao de LATENCIA
4. Restricao de DADOS (Sensores Corrompidos)
5. Restricao ALGORITMICA (XOR)
Escolha:
```

Figura 2 – Menu do teste benchmark.

```
=====
LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintotica)
=====
0. Baseline (Condicoes Ideais)
1. Restricao de MEMORIA (Max 500 nos)
2. Restricao de PROCESSAMENTO
3. Restricao de LATENCIA
4. Restricao de DADOS (Sensores Corrompidos)
5. Restricao ALGORITMICA (XOR)
Escolha: 0

[INFO] Iniciando avaliacao O(N), O(log N) e O(1) em Multiplas Cargas...
=====
(N) | T. Medio Ins. HASH | T. Medio Ins. AVL | T. Medio Busca HASH | T. Medio Busca AVL | OVERHEAD RAM AVL
=====
1000 | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.03 MB
5000 | 0.000000 seg | 0.000001 seg | 0.000000 seg | 0.000000 seg | 0.15 MB
10000 | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.31 MB
25000 | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.76 MB
50000 | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.000000 seg | 1.53 MB
100000 | 0.000000 seg | 0.000000 seg | 0.000000 seg | 0.000000 seg | 3.05 MB
=====

ANALISE DE COMPLEXIDADE ESTRUTURAL DA ULTIMA CARGA (N = 100000)
=====
Estrutura | Consumo RAM | Tempo Total | Justificativa de Arquitetura
=====
AVL (Calculo O(N)) | 3.05 MB | 106.8720 seg | Acesso Linear lento, ideal para insercoes dinamicas.
AVL Opt (Calc O(1)) | 3.81 MB | 0.0010 seg | Trade-off: Gasta 8 Bytes extras p/no pela Busca Imediata.
Hash Table O(1) | 2.29 MB | - | Excepcional para ID/Datas exatas, inutil para Range Query.
KD-Tree Vetorial | 2.29 MB | - | Obrigatoria para buscas de Aceleracao(X,Y,Z) em O(log N).
Segment Tree O(log) | 0.06 MB | - | Alocacao array estatico fixa, perfeita p/ medias moveis.
=====
```

Figura 3 – Opção 0 selecionada.

LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintotica)					
0. Baseline (Condicoes Ideais)					
1. Restricao de MEMORIA (Max 500 nos)					
2. Restricao de PROCESSAMENTO					
3. Restricao de LATENCIA					
4. Restricao de DADOS (Sensores Corrompidos)					
5. Restricao ALGORITMICA (XOR)					
Escolha: 1					
[INFO] Iniciando avaliacao O(N), O(log N) e O(1) em Multiplas Cargas...					
(N)	T. Medio Ins. HASH	T. Medio Ins. AVL	T. Medio Busca HASH	T. Medio Busca AVL	OVERHEAD RAM AVL
1000	0.000000 seg	0.000001 seg	0.000000 seg	0.000000 seg	0.03 MB
5000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.15 MB
10000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.31 MB
25000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.76 MB
50000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	1.53 MB
100000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	3.05 MB
ANALISE DE COMPLEXIDADE ESTRUTURAL DA ULTIMA CARGA (N = 100000)					
Estrutura	Consumo RAM	Tempo Total	Justificativa de Arquitetura		
AVL (Calculo O(N))	3.05 MB	111.7420 seg	Acesso Linear lento, ideal para insercoes dinamicas.		
AVL Opt (Calc O(1))	0.02 MB	0.0000 seg	Trade-off: Gasta 8 Bytes extras p/no pela Busca Imediata.		
Hash Table O(1)	2.29 MB	-	Excepcional para ID/Datas exatas, inutil para Range Query.		
KD-Tree Vetorial	2.29 MB	-	Obrigatoria para buscas de Aceleracao(X,Y,Z) em O(log N).		
Segment Tree O(log)	0.06 MB	-	Alocacao array estatico fixa, perfeita p/ medias moveis.		

Figura 4 – Opção 1 selecionada.

LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintotica)					
0. Baseline (Condicoes Ideais)					
1. Restricao de MEMORIA (Max 500 nos)					
2. Restricao de PROCESSAMENTO					
3. Restricao de LATENCIA					
4. Restricao de DADOS (Sensores Corrompidos)					
5. Restricao ALGORITMICA (XOR)					
Escolha: 2					
[INFO] Iniciando avaliacao O(N), O(log N) e O(1) em Multiplas Cargas...					
(N)	T. Medio Ins. HASH	T. Medio Ins. AVL	T. Medio Busca HASH	T. Medio Busca AVL	OVERHEAD RAM AVL
1000	0.000011 seg	0.000011 seg	0.000000 seg	0.000001 seg	0.03 MB
5000	0.000011 seg	0.000011 seg	0.000000 seg	0.000000 seg	0.15 MB
10000	0.000010 seg	0.000011 seg	0.000000 seg	0.000000 seg	0.31 MB
25000	0.000011 seg	0.000011 seg	0.000000 seg	0.000000 seg	0.76 MB
50000	0.000011 seg	0.000011 seg	0.000000 seg	0.000000 seg	1.53 MB
100000	0.000010 seg	0.000011 seg	0.000000 seg	0.000000 seg	3.05 MB
ANALISE DE COMPLEXIDADE ESTRUTURAL DA ULTIMA CARGA (N = 100000)					
Estrutura	Consumo RAM	Tempo Total	Justificativa de Arquitetura		
AVL (Calculo O(N))	3.05 MB	102.3800 seg	Acesso Linear lento, ideal para insercoes dinamicas.		
AVL Opt (Calc O(1))	3.81 MB	0.0000 seg	Trade-off: Gasta 8 Bytes extras p/no pela Busca Imediata.		
Hash Table O(1)	2.29 MB	-	Excepcional para ID/Datas exatas, inutil para Range Query.		
KD-Tree Vetorial	2.29 MB	-	Obrigatoria para buscas de Aceleracao(X,Y,Z) em O(log N).		
Segment Tree O(log)	0.06 MB	-	Alocacao array estatico fixa, perfeita p/ medias moveis.		

Figura 5 – Opção 2 selecionada.

LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintotica)					
0. Baseline (Condições Ideais)					
1. Restrição de MEMÓRIA (Max 500 nos)					
2. Restrição de PROCESSAMENTO					
3. Restrição de LATÊNCIA					
4. Restrição de DADOS (Sensores Corrompidos)					
5. Restrição ALGORÍTMICA (XOR)					
Escolha: 3					
[INFO] Iniciando avaliação $O(N)$, $O(\log N)$ e $O(1)$ em Múltiplas Cargas...					
(N)	T. Medio Ins. HASH	T. Medio Ins. AVL	T. Medio Busca HASH	T. Medio Busca AVL	OVERHEAD RAM AVL
1000	0.000010 seg	0.000008 seg	0.000000 seg	0.000000 seg	0.03 MB
5000	0.000009 seg	0.000009 seg	0.000000 seg	0.000000 seg	0.15 MB
10000	0.000009 seg	0.000009 seg	0.000000 seg	0.000000 seg	0.31 MB
25000	0.000009 seg	0.000009 seg	0.000000 seg	0.000000 seg	0.76 MB
50000	0.000008 seg	0.000009 seg	0.000000 seg	0.000000 seg	1.53 MB
100000	0.000009 seg	0.000009 seg	0.000000 seg	0.000000 seg	3.05 MB
ANÁLISE DE COMPLEXIDADE ESTRUTURAL DA ÚLTIMA CARGA (N = 100000)					
Estrutura	Consumo RAM	Tempo Total	Justificativa de Arquitetura		
AVL (Cálculo $O(N)$)	3.05 MB	99.9660 seg	Acesso Linear lento, ideal para inserções dinâmicas.		
AVL Opt (Calc $O(1)$)	3.81 MB	0.0000 seg	Trade-off: Gasta 8 Bytes extras p/no pela Busca Imediata.		
Hash Table $O(1)$	2.29 MB	-	Excepcional para ID/Datas exatas, inútil para Range Query.		
KD-Tree Vetorial	2.29 MB	-	Obrigatória para buscas de Aceleração(X,Y,Z) em $O(\log N)$.		
Segment Tree $O(\log)$	0.06 MB	-	Alocação array estático fixa, perfeita p/ médias móveis.		

Figura 6 – Opção 3 selecionada.

LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintotica)					
0. Baseline (Condições Ideais)					
1. Restrição de MEMÓRIA (Max 500 nos)					
2. Restrição de PROCESSAMENTO					
3. Restrição de LATÊNCIA					
4. Restrição de DADOS (Sensores Corrompidos)					
5. Restrição ALGORÍTMICA (XOR)					
Escolha: 4					
[INFO] Iniciando avaliação $O(N)$, $O(\log N)$ e $O(1)$ em Múltiplas Cargas...					
(N)	T. Medio Ins. HASH	T. Medio Ins. AVL	T. Medio Busca HASH	T. Medio Busca AVL	OVERHEAD RAM AVL
1000	0.000001 seg	0.000000 seg	0.000001 seg	0.000000 seg	0.03 MB
5000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.15 MB
10000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.31 MB
25000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.76 MB
50000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	1.53 MB
100000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	3.05 MB
ANÁLISE DE COMPLEXIDADE ESTRUTURAL DA ÚLTIMA CARGA (N = 100000)					
Estrutura	Consumo RAM	Tempo Total	Justificativa de Arquitetura		
AVL (Cálculo $O(N)$)	3.05 MB	127.5270 seg	Acesso Linear lento, ideal para inserções dinâmicas.		
AVL Opt (Calc $O(1)$)	3.81 MB	0.0010 seg	Trade-off: Gasta 8 Bytes extras p/no pela Busca Imediata.		
Hash Table $O(1)$	2.29 MB	-	Excepcional para ID/Datas exatas, inútil para Range Query.		
KD-Tree Vetorial	2.29 MB	-	Obrigatória para buscas de Aceleração(X,Y,Z) em $O(\log N)$.		
Segment Tree $O(\log)$	0.06 MB	-	Alocação array estático fixa, perfeita p/ médias móveis.		

Figura 6 – Opção 4 selecionada.

LABORATORIO DE BENCHMARKS AUTOMATIZADO (Escalabilidade e Complexidade Assintótica)					
0. Baseline (Condições Ideais)					
1. Restrição de MEMÓRIA (Max 500 nos)					
2. Restrição de PROCESSAMENTO					
3. Restrição de LATÊNCIA					
4. Restrição de DADOS (Sensores Corrompidos)					
5. Restrição ALGORÍTMICA (XOR)					
Escolha: 5					
[INFO] Iniciando avaliação $O(N)$, $O(\log N)$ e $O(1)$ em Múltiplas Cargas...					
(N)	T. Medio Ins. HASH	T. Medio Ins. AVL	T. Medio Busca HASH	T. Medio Busca AVL	OVERHEAD RAM AVL
1000	0.000000 seg	0.000001 seg	0.000000 seg	0.000000 seg	0.03 MB
5000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.15 MB
10000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.31 MB
25000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	0.76 MB
50000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	1.53 MB
100000	0.000000 seg	0.000000 seg	0.000000 seg	0.000000 seg	3.05 MB
ANÁLISE DE COMPLEXIDADE ESTRUTURAL DA ÚLTIMA CARGA (N = 100000)					
Estrutura	Consumo RAM	Tempo Total	Justificativa de Arquitetura		
AVL (Cálculo $O(N)$)	3.05 MB	112.4120 seg	Acesso Linear lento, ideal para inserções dinâmicas.		
AVL Opt (Calc $O(1)$)	3.81 MB	0.0000 seg	Trade-off: Gasta 8 Bytes extras p/no pela Busca Imediata.		
Hash Table $O(1)$	2.29 MB	-	Excepcional para ID/Dados exatas, inútil para Range Query.		
KD-Tree Vetorial	2.29 MB	-	Obrigatória para buscas de Aceleração(X,Y,Z) em $O(\log N)$.		
Segment Tree $O(\log)$	0.06 MB	-	Alocação array estático fixa, perfeita p/ médias móveis.		

Figura 7 – Opção 5 selecionada.

9.1. Desempenho em Cenário Ideal (Baseline)

Em condições ideais com 100.000 registros, inserções e buscas foram instantâneas (0.000000s). A árvore AVL padrão ($O(N)$) gastou 3.05 MB e levou 109.8270s para processar a carga. A AVL otimizada ($O(1)$) zerou este tempo de cálculo (0.0000s), exigindo em troca um consumo maior de 3.81 MB. As demais estruturas mantiveram um consumo eficiente: Hash e KD-Tree com 2.29 MB cada, e a Segment Tree com apenas 0.06 MB.

9.2. Restrição de Memória

A simulação de escassez comprovou a eficácia da poda de memória. O consumo da AVL otimizada despencou drasticamente de 3.81 MB para apenas 0.02 MB, mantendo a execução em perfeitos 0.0000s. Em contraste, a AVL padrão $O(N)$ não reduziu seu consumo (3.05 MB) e registrou um tempo total de 111.7420s, demonstrando a superioridade da versão otimizada em hardwares limitados.

9.3. Restrição de processamento

O estrangulamento da CPU elevou os tempos médios unitários de inserção da Hash e da AVL para a faixa de 0.000010s a 0.000011s. Mesmo sob essa lentidão forçada, a AVL otimizada manteve resiliência e concluiu os cálculos em 0.0000s. A AVL padrão finalizou em 102.3800s, confirmando que

a limitação de processador pune o custo individual de alocação, mas não trava o fluxo contínuo do sistema.

9.4. Restrição de Latência

A introdução de atrasos (simulando gargalos de I/O) travou a inserção média entre 0.000009s e 0.000010s. Curiosamente, a AVL padrão ($O(N)$) obteve neste cenário seu processamento em lote mais rápido: 99.9660s. A AVL otimizada novamente cravou 0.0000s. Isso prova que a alta latência afeta puramente o gargalo de recepção inicial dos dados, sem prejudicar o recálculo e a ordenação interna das árvores.

9.5. Restrição de dados (Sensores Corrompidos)

Injetar ruídos e dados impossíveis exigiu o esforço máximo do algoritmo para filtrar falsos positivos. Isso resultou no pior desempenho da AVL padrão ($O(N)$), que saltou para 127.5270s para processar a carga total. Além disso, este foi o único cenário adverso onde a AVL otimizada não conseguiu zerar totalmente o relógio, registrando um leve micro-atraso de 0.0010s devido ao alto volume de validações lógicas.

9.6. Restrição Algorítmica (XOR)

A indução de operações lógicas complexas (XOR) não conseguiu quebrar o espalhamento dos dados. Os tempos médios unitários voltaram a 0.000000s e o uso de memória espelhou exatamente o Baseline (3.05 MB na AVL padrão e 3.81 MB na otimizada). A árvore padrão fechou em 112.4120s e a otimizada em 0.0000s, provando que o dimensionamento matemático correto do Hash blinda o sistema contra degradações intencionais.

9.7. Comparação entre AVL Tradicional e AVL Otimizada

A comparação entre a Árvore AVL Tradicional (cálculo em $O(N)$) e a AVL Otimizada (cálculo em $O(1)$) faz a comparação entre tempo de processamento e consumo de memória. A versão otimizada demonstrou um desempenho bom nos testes, concluindo as operações em 0.0000 segundos, em troca teve um ligeiro aumento no consumo de RAM (de 3.05 MB para 3.81 MB) devido ao armazenamento de 8 bytes extra por nó para cálculos pré-computados. Em comparação, a AVL tradicional economiza na alocação estrutural básica, mas deixa de lado drasticamente a velocidade, demorando entre 99 e 127 segundos para processar os mesmos 100.000 registros por conta da necessidade de efetuar travessias lineares lentas constantes. O diferencial da AVL Otimizada revelou-se na restrição de memória (limite de 500 nós). Graças à sua lógica de corte dinâmico associada ao cálculo imediato, o consumo desta versão caiu drasticamente para apenas 0.02 MB, mantendo a execução instantânea, enquanto a AVL tradicional não conseguiu aliviar o seu consumo (mantendo os 3.05 MB) e ultrapassou os 111 segundos de processamento.

10. Discussão dos Resultados

Os resultados obtidos no laboratório de benchmarks mostram a importância de adequar a escolha algorítmica aos recursos e limitações do ambiente de execução. Inicialmente, é fundamental justificar que a comparação direta de tempo de busca entre todas as cinco estruturas implementadas não é metodologicamente válida de forma isolada, pois elas resolvem problemas distintos e não competem diretamente. A KD-Tree foi desenhada especificamente para buscas espaciais multidimensionais dos eixos X, Y e Z, a Segment Tree opera com eficiência máxima em consultas de intervalos e médias móveis, e a Tabela Hash é voltada estritamente para buscas exatas por data. Por essa razão, a análise de escalabilidade e o comparativo direto de desempenho focaram prioritariamente no embate entre a AVL Padrão e a AVL Otimizada, nas quais a função, o cenário de ordenação e a necessidade de recálculos são equiparáveis, garantindo assim o rigor metodológico da avaliação empírica.

Neste contexto, a utilização do dataset com 11,5 milhões de amostras reais provou-se adequada e amplamente suficiente para evidenciar as diferenças de complexidade assintótica durante os testes de estresse.

11. Conclusão

O desenvolvimento deste sistema de monitoramento biométrico para enfermeiras simulou com sucesso um ambiente de IoT crítico, aplicando de forma integrada e prática o uso estruturado de dados em massa. O projeto cumpriu com todos os objetivos propostos e confirmou na prática que o design de um software de alto desempenho moderno exige a coexistência de múltiplas estruturas.

12. Referências

RAVAL, Priyank. Nurse Stress Prediction Wearable Sensors. Kaggle, 2023. Conjunto de dados. Disponível em: <<https://www.kaggle.com/datasets/priyankraval/nurse-stress-prediction-wearable-sensors/data>>. Acesso em: 27 jun. 2026.

TELAVITA. Diferença de Burnout e estresse. Blog Telavita, [s.d.]. Disponível em: <<https://www.telavita.com.br/blog/diferenca-de-burnout-e-estresse/>>. Acedido em: 28 jun. 2026.